

Developer Guide — Decoder

From first project to a fully featured DCC decoder

Covers the library on any platform; the TI MSPM0G3507 LaunchPad is the worked example

Generated 2026-09-23 from the repository sources by documentation/guides/build_pdfs.py

Table of Contents

1. Introduction	3
1.1 What the Library Does (and Does Not Do)	3
1.2 Platform Support	3
2. DCC Decoder Concepts	4
2.1 Signal Reception	4
2.2 Bit Classification	4
2.3 Packet Structure	4
2.4 Address Matching	4
2.5 Configuration Variables	4
3. Project File Structure	6
4. dcc_user_config.h in Depth	7
5. Initialization — decoder.c	8
5.1 The dcc_config_t Struct (Decoder Fields)	8
5.2 Edge Timestamp Ring Buffer	8
5.3 Setup and Main Loop	9
6. ISR Architecture	10
6.1 GPIO Edge Interrupt	10
6.2 100 ms Tick	10
7. Implementing the Drivers	11
8. The Bit Decoder	12
9. The Packet Decoder	13
10. CV Storage, Lock, and CV 29	14
11. Service Mode (Decoder Side)	15
12. RailCom Replies	16
13. Fail-Safe	17
14. Porting to a New MCU	18
15. Unit Testing	19
16. Troubleshooting	20

1. Introduction

OpenDccCLib is a C library that implements the NMRA DCC protocol for microcontrollers. This guide covers the decoder role: receiving DCC from the track, recognizing the commands addressed to this decoder, and acting on speed, function, accessory, and CV programming commands. It allocates no dynamic memory and needs no operating system; a main loop and one edge interrupt are enough.

1.1 What the Library Does (and Does Not Do)

The library classifies one and zero bits from edge timing, assembles packets and validates the XOR byte, matches addresses against the CVs, dispatches commands to your callbacks from the main loop, wraps CV storage with the decoder lock and factory reset, answers service-mode programming with the 6 ms ACK pulse, runs the S-9.2.4 packet time-out, and, when wired, transmits RailCom replies during the cutout.

It does not include motor control, lighting, servos, sound, or persistent storage. You supply those through function pointers in one `dcc_config_t` struct, and the library calls them.

1.2 Platform Support

Processor / Board	Example IDE / Toolchain
TI MSPM0G3507 LaunchPad	Code Composer Studio / Theia (the shipped example)
RP2040 / RP2350	Arduino IDE, PlatformIO, Pico SDK
STM32 (F4 and others)	STM32CubeIDE
ESP32	Arduino IDE, PlatformIO
Any ARM Cortex-M	GCC + Makefile

The only hardware a decoder needs is a GPIO edge interrupt and a microsecond timestamp. RailCom transmit additionally needs a cycle-accurate microsecond delay.

2. DCC Decoder Concepts

2.1 Signal Reception

The track carries a bipolar square wave. The decoder rectifies it for power and converts it to a logic-level signal through an optocoupler or divider. Every transition is a half-bit boundary; the decoder measures the time between transitions.

2.2 Bit Classification

Interval between edges	Classification	Define (dcc_defines.h)
< 80 μ s	One-bit half (58 μ s nominal)	DCC_DECODER_HALF_BIT_THRESHOLD_US = 80
80 μ s ... 10 000 μ s	Zero-bit half (100 μ s nominal; NMRA allows stretching)	
$\geq$ 10 000 μ s	No signal; the decoder resets to preamble search	DCC_DECODER_HALF_BIT_MAX_US = 10000

Two consecutive halves of the same class make one bit. A short half followed by a long one is invalid and restarts the preamble search.

2.3 Packet Structure

A valid preamble is at least `DCC_PREAMBLE_BITS_DECODER_MIN` = 10 one-bits (command stations send 14 or more). A zero start bit begins each byte; a one end bit closes the packet. The last byte is the XOR of all preceding bytes; a mismatch discards the packet.

2.4 Address Matching

The library reads its addressing CVs at `DccConfig_initialize()` and again whenever one of them is written.

CV	Purpose
CV 1	Primary (short) address, 1–127
CV 17–18	Extended (long) address, 128–10239
CV 29 bit 5	0 = use CV 1, 1 = use CV 17–18
CV 19	Consist address; a consist speed command is accepted when it matches
CV 513, 521	Accessory decoder address (low 6 bits, high 3 bits)
CV 541	Accessory configuration: bit 7 accessory decoder, bit 6 output-address mode, bit 5 extended

Broadcast (address 0) is always accepted.

2.5 Configuration Variables

CV	Name	Note
1	Primary address	Demo default 3

CV	Name	Note
2–6	Vstart, acceleration, deceleration, Vhigh, Vmid	Application use
7	Manufacturer version	Read-only
8	Manufacturer ID	Read-only; writing 8 triggers factory_reset
11	Packet time-out	Fail-safe, in 100 ms units; 0 disables
15 / 16	Decoder lock	Writes are refused unless CV 15 equals CV 16
17–18	Extended address	
19	Consist address	
28	RailCom configuration	bit 0 Channel 1, bit 1 Channel 2
29	Configuration	See DCC_CV29_*_BIT; bit 6 is reserved and forced to 0
31 / 32	Indexed CV page	Selects the page behind CVs 257–512
257–512	Indexed window	Served by cv_read_indexed / cv_write_indexed when wired

3. Project File Structure

decoder/	<- your project folder
decoder.c	<- main entry: config struct, edge ISR, ring buffer, main loop
dcc_user_config.h	<- REQUIRED: role flags and limits
decoder_command_parser.c/h	<- demo UART command-line interface
application_callbacks/	
callbacks_dcc.c/h	<- CV storage stub, CV29 feature mask, command callbacks
application_drivers/	
ti_driverlib_dcc_driver.c/h	<- timestamp, lock, RailCom delay
ti_driverlib_uart_driver.c/h	<- UART I/O
ack_pulse_driver.c/h	<- ACK current load
dcc_lib/ -> ../../../../src/dcc	<- library core (symlink; do not edit)
dcc_config.h/c	- config struct, lifecycle, wiring of every module
dcc_bit_decoder.h/c	- edge timing -> bits -> bytes
dcc_packet_decoder.h/c	- XOR, address match, dispatch queue
dcc_cv_storage.h/c	- CV access, decoder lock, factory reset, indexed CVs
dcc_failsafe.h/c	- CV 11 packet time-out
dcc_railcom_decoder.h/c	- RailCom transmit engine (bit-bang)
dcc_railcom_utilities.h/c	- 4/8 code words
dcc_application_decoder_cv.h/c	- CV read/write API
dcc_application_decoder_railcom.h/c	- RailCom reply API
dcc_application_accessory_decoder_railcom.h/c	- accessory-decoder RailCom API

4. dcc_user_config.h in Depth

```
#define DCC_COMPILE_DECODER
#define DCC_COMPILE_RAILCOM // RailCom transmit; comment out to strip
// #define DCC_COMPILE_ACCESSORY_DECODER // accessory-decoder RailCom (SRQ, status)

#define USER_DEFINED_DCC_DECODER_MAX_FUNCTIONS 29 // F0-F28; raise for F29-F68
#define USER_DEFINED_DCC_DECODER_PACKET_QUEUE_DEPTH 8 // deferred dispatch queue, >= 2
```

Define	Meaning
DCC_COMPILE_DECODER	Bit decoder, packet decoder, CV storage, fail-safe
DCC_COMPILE_RAILCOM	Compiles the RailCom transmit engine; active only when railcom_tx_pin_set is wired
DCC_COMPILE_ACCESSORY_DECODER	Accessory-decoder RailCom replies (SRQ, status, time, error)
USER_DEFINED_DCC_DECODER_MAX_FUNCTIONS	Highest function number the decoder tracks
USER_DEFINED_DCC_DECODER_PACKET_QUEUE_DEPTH	Packets that can wait for main-loop dispatch; one slot is reserved, so ≥ 2

A decoder project must not define DCC_COMPILE_COMMAND_STATION unless it really is both, as a booster or repeater would be.

5. Initialization — decoder.c

5.1 The dcc_config_t Struct (Decoder Fields)

Field	Required?	Purpose
lock_shared_resources, unlock_shared_resources, get_timestamp_usec	REQUIRED	Platform: interrupt mask and microsecond clock
cv_read(cv, *value), cv_write(cv, value)	REQUIRED	Persistent CV storage
cv29_apply_supported_features(flags)	REQUIRED	On every CV 29 write, clear the feature bits this product does not implement; the library stores what you leave set
factory_reset()	optional	Called when a write targets CV 8
cv_read_indexed, cv_write_indexed	optional	Serve CVs 257–512 for the page in CV 31:32
railcom_tx_pin_set(high)	optional	Drive the RailCom current source. NULL = no RailCom transmit
railcom_delay_us(us)	required with the above	Cycle-accurate busy-wait for the 4 μ s bit period; a 1 μ s-granular timer is not accurate enough
on_railcom_request(instruction, count, *out)	optional	Fill a Channel 2 reply for a command addressed to this decoder; return DATA, ACK, BUSY, NACK or NONE
start_ack_pulse(), stop_ack_pulse()	optional	Service-mode ACK current load; the library times the 6 ms
on_*_command(...), on_failsafe_entered/exited	optional	Application notifications, main loop

```
const dcc_config_t dcc_config = {
    .lock_shared_resources = &TI_DccDriver_lock_shared_resources,
    .unlock_shared_resources = &TI_DccDriver_unlock_shared_resources,
    .get_timestamp_usec = &TI_DccDriver_get_timestamp_usec,

    .cv_read = &CallbacksDcc_cv_read,          /* RAM stub in the demo */
    .cv_write = &CallbacksDcc_cv_write,
    .factory_reset = &CallbacksDcc_factory_reset,
    .cv_read_indexed = &CallbacksDcc_cv_read_indexed,
    .cv_write_indexed = &CallbacksDcc_cv_write_indexed,
    .cv29_apply_supported_features = &CallbacksDcc_cv29_apply_supported_features,

    .railcom_tx_pin_set = NULL,                /* no current source on the demo board */
    .railcom_delay_us = &TI_DccDriver_railcom_delay_us,

    .start_ack_pulse = &AckPulseDriver_start,
    .stop_ack_pulse = &AckPulseDriver_stop,

    .on_speed_command = &CallbacksDcc_on_speed_command,
    /* ... the other command callbacks ... */
    .on_failsafe_entered = &CallbacksDcc_on_failsafe_entered,
    .on_failsafe_exited = &CallbacksDcc_on_failsafe_exited,
};
```

5.2 Edge Timestamp Ring Buffer

The GPIO interrupt only captures a timestamp into a power-of-two ring; the main loop drains it into `DccConfig_decoder_edge_isr()`. That keeps the interrupt under a microsecond so it never overruns the 58 μ s half-bit, and it means bit decoding runs in the main loop on the demo. Calling `DccConfig_decoder_edge_isr()` directly from the interrupt is equally valid on a faster part.

```
#define EDGE_BUF_SIZE 256 /* power of two */
static volatile uint32_t _edge_buf[EDGE_BUF_SIZE];
static volatile uint16_t _edge_head, _edge_tail;

void GROUP1_IRQHandler(void) { /* PB1 main, PB4 service, PB17 selects */
    ...
    _edge_buf[_edge_head] = TI_DccDriver_get_timestamp_usec();
    _edge_head = (_edge_head + 1) & (EDGE_BUF_SIZE - 1);
}

static void _drain_edge_buffer(void) {
    while (_edge_tail != _edge_head) {
        DccConfig_decoder_edge_isr(_edge_buf[_edge_tail]);
        _edge_tail = (_edge_tail + 1) & (EDGE_BUF_SIZE - 1);
    }
}
```

5.3 Setup and Main Loop

```
int main(void) {
    SYSCFG_DL_init();
    NVIC_EnableIRQ(GPIOB_INT_IRQn);
    TI_DccDriver_initialize();
    TI_UartDriver_initialize();
    AckPulseDriver_initialize();
    CallbacksDcc_initialize(); /* CV defaults: CV1 = 3, CV29 = 0x06 */
    DccConfig_initialize(&dcc_config); /* reads the addressing CVs */
    DecoderCommandParser_initialize();

    while (1) {
        _drain_edge_buffer(); /* edges -> bit decoder */
        DccConfig_run(); /* packet dispatch, callbacks, ACK timing, fail-safe */
        CallbacksDcc_drain(); /* RECV lines -> UART */
        TI_UartDriver_echo_process();
        DecoderCommandParser_process();
    }
}
```

If the ring buffer overflows, edges are dropped silently and packets are lost. Keep every call in the loop non-blocking.

6. ISR Architecture

6.1 GPIO Edge Interrupt

Fires on every rising and falling edge of the DCC input. The demo has two inputs on one port, PB1 for the main track and PB4 for the service track, and a track-select input on PB17 chooses which one is accepted. During a RailCom transmit the library masks this interrupt through `lock_shared_resources`, so the decoder's own current pulses cannot trigger it.

6.2 100 ms Tick

The library's `DccConfig_100ms_timer_tick()` belongs to the command-station role; the decoder needs no periodic tick. The demo's SysTick only blinks the heartbeat LED. Fail-safe timing uses `get_timestamp_usec` from `DccConfig_run()`.

7. Implementing the Drivers

Function	Context	Contract
lock_shared_resources / unlock_shared_resources	any	Mask and restore interrupts; must cover the DCC edge interrupt
get_timestamp_usec	ISR	Free-running microsecond counter; monotonic, wrap is fine
cv_read / cv_write	main loop	Persistent storage. The demo uses a RAM array that resets at power-up; use Flash or EEPROM in a product
start_ack_pulse / stop_ack_pulse	main loop	Switch the ACK current load; the library calls stop after DCC_ACK_PULSE_DURATION_US = 6000 μ s
railcom_tx_pin_set	ISR	Set the RailCom line to a UART level, true = mark
railcom_delay_us	ISR	Spin exactly that many microseconds at 4 μ s resolution (a DWT cycle counter, for example)

8. The Bit Decoder

`DccConfig_decoder_edge_isr(timestamp)` computes the interval since the previous edge and classifies it as in section 2.2. Halves pair into bits; ten or more consecutive ones arm the preamble; a zero then starts byte assembly. Each completed byte and the finished packet are handed to the packet decoder through interface pointers, never by direct call.

9. The Packet Decoder

Completed packets are queued (`USER_DEFINED_DCC_DECODER_PACKET_QUEUE_DEPTH`) and dispatched from `DccConfig_run()`. This deferral keeps long handlers such as CV writes out of the edge path, and it is why every callback runs in main-loop context. For each packet the decoder validates the XOR, matches the address, then parses the instruction and calls the matching callback. A packet addressed to this decoder also re-arms the fail-safe timer and, when RailCom is compiled in, updates the reply engine's address.

Instruction	Callback
Speed 14/28/128	<code>on_speed_command(address, speed, direction, mode);</code> mode is a <code>dcc_speed_mode_enum</code>
Emergency stop	<code>on_emergency_stop_command(address)</code>
Functions F0–F68	<code>on_function_command(address, function_number, state)</code>
Basic / extended accessory	<code>on_accessory_basic_command(board, pair, activate),</code> <code>on_accessory_extended_command(address, aspect)</code>
CV write / verify / bit, main track or service track	<code>on_cv_write_command(cv, value, service_mode),</code> <code>on_cv_verify_command(...),</code> <code>on_cv_bit_command(cv, bit, value, service_mode)</code>
Consist	<code>on_consist_command(address, consist_address, direction_normal)</code>
Binary state short / long	<code>on_binary_state_short_command(address, state, active),</code> <code>on_binary_state_long_command(...)</code>
Analog function	<code>on_analog_function_command(address, output, value)</code>

10. CV Storage, Lock, and CV 29

All CV traffic goes through `dcc_cv_storage`, which wraps your `cv_read` / `cv_write`:

- **Decoder lock.** A write is refused unless CV 15 equals CV 16; CV 15 and CV 16 themselves are always writable.
- **Factory reset.** A write to CV 8 calls `factory_reset()` instead of storing.
- **Indexed CVs.** CVs 257–512 are routed to `cv_read_indexed` / `cv_write_indexed` with the page from CV 31:32, when those hooks are wired.
- **CV 29 feature mask.** On every CV 29 write the library forces the reserved bit 6 clear, decodes the byte into a `dcc_cv29_flags_t` (direction reversed, 28/128 steps, analog conversion, RailCom, speed table, extended address, accessory), and calls `cv29_apply_supported_features()`. Clear the flags your product does not implement; per S-9.2.2 an unsupported feature bit must never be settable, and only the application knows what it supports.

The application API is `DccApplicationDecoderCv_read`, `_write` and `_is_locked`.

11. Service Mode (Decoder Side)

On the programming track the library recognizes the service-mode sequences and handles them: a write stores through `cv_write` and answers with the ACK pulse; a verify compares through `cv_read` and answers only on a match; bit operations do the same at bit level. Your CV callbacks fire afterwards with `service_mode` true, so you can tell programming-track writes from operations-mode ones.

12. RailCom Replies

With `railcom_tx_pin_set` and `railcom_delay_us` wired, the library transmits during the cutout that follows a packet addressed to this decoder, bit-banging 4/8-encoded bytes at 250 kbaud with `DCC_RAILCOM_TX_BIT_US = 4 µs` bits, after a `DCC_RAILCOM_TX_BLANK_US = 80 µs` blanking delay. Interrupts are masked for the transmit window through `lock_shared_resources`.

- **Channel 1** carries the address broadcast, alternating ADR1 and ADR2, sent automatically.
- **Channel 2** is yours: `on_railcom_request()` is called as soon as a command addressed to this decoder is recognized, before its XOR byte, and returns a reply status. Return `DCC_RAILCOM_REPLY_DATA` with a `dcc_railcom_response_t` (id plus up to six bytes), or ACK, BUSY, NACK, or NONE.
- The helper API `DccApplicationDecoderRailcom_send_*` builds common replies: address feedback, POM response, dynamic data, track-search response, CV auto-transfer, ACK, NACK, raw.
- Accessory decoders use `DccApplicationAccessoryDecoderRailcom_*`: service request (SRQ), status, time and error reports, and the stop-command and cutout hooks.

The shipped decoder board has no current-source circuit, so the demo leaves `railcom_tx_pin_set` NULL and no reply is transmitted. The engine and encoders are unit-tested; the on-track side is a known open item in the compliance overview.

13. Fail-Safe

S-9.2.4 requires a decoder to stop everything when no packet addressed to it arrives within its time-out. The library implements this in `dcc_failsafe`: CV 11 times `DCC_FAILSAFE_CV11_UNIT_US = 100 000 µs`, polled from `DccConfig_run()`, re-armed by every addressed packet. CV 11 = 0 disables it. `on_failsafe_entered()` is where you stop the motor and drop the outputs; `on_failsafe_exited()` fires when an addressed packet returns.

14. Porting to a New MCU

1. Copy the decoder example project.
2. Rewrite `ti_driverlib_dcc_driver.c/h`: interrupt mask, microsecond timestamp, and a cycle-accurate `railcom_delay_us` if you transmit RailCom.
3. Replace `ack_pulse_driver.c/h` with your current-load control, or leave both ACK hooks NULL.
4. Replace the CV stub with Flash or EEPROM access.
5. Wire the edge interrupt to capture timestamps, and the ring-buffer drain to `DccConfig_decoder_edge_isr()`.
6. Build and test against a known-good command station; the RECV lines over UART show every decoded command.

15. Unit Testing

Decoder-role tests, GoogleTest with mocked drivers, run with the rest of the suite: `cd test && make`. At generation time the whole suite is 29 binaries, 1130 tests, 0 failures, with 95.3 % line coverage.

Test file	What it tests
<code>dcc_bit_decoder_Test</code>	Edge classification, thresholds, preamble, noise and time-out recovery
<code>dcc_packet_decoder_Test</code>	XOR, address matching for every address type, instruction dispatch, queue
<code>dcc_cv_storage_Test</code> , <code>dcc_application_decoder_cv_Test</code>	Lock, factory reset, indexed CVs, CV 29 feature mask
<code>dcc_failsafe_Test</code>	CV 11 time-out, enter and exit
<code>dcc_railcom_decoder_Test</code> , <code>dcc_railcom_utilities_Test</code> , <code>dcc_application_decoder_railcom_Test</code>	Reply engine, 4/8 encoding against the spec table, reply builders
<code>dcc_application_accessory_decoder_railcom_Test</code>	Accessory SRQ, status, time and error replies

16. Troubleshooting

Symptom	Likely cause and fix
No packets decoded	Input wiring, edge interrupt not enabled, or the ring buffer is not drained
Packets decode but callbacks stay silent	Address mismatch; check CV 1 and CV 29 bit 5, or try the broadcast address
Frequent XOR failures	Noisy input; clean the signal to logic levels and check the threshold circuit
Command station reports NO ACK	ACK current load not wired or too weak; verify a 6 ms pulse on the ACK pin
Fail-safe trips at once	No DCC reaching the decoder, or CV 11 set very small (units of 100 ms)
Functions work but speed does not	CV 29 bit 1 speed-step setting does not match the command station
Address lost after power cycle	CV storage is RAM in the demo; use non-volatile storage
No RailCom reply seen	<code>railcom_tx_pin_set</code> is NULL, or <code>railcom_delay_us</code> is not cycle-accurate at 4 μ s

Library source: github.com/JimKueneman/OpenDccCLib. NMRA DCC standards: www.nmra.org/dcc.